

The Economics of LLM Inference

A Roofline Analysis of Transformers on GPU Clusters

Notes from the Dwarkesh Patel blackboard lecture with Reiner Pope (CEO, MatX; formerly TPU architecture at Google)

Compiled June 2026

Motivating question. Providers offer “Fast Mode” at $\sim 6\times$ the price for $\sim 2.5\times$ the speed. Why does paying more buy speed? Could a “Slow Mode” be much cheaper? The answer is almost entirely about *batch size*, and a simple roofline model of one rack of GPUs is enough to derive it — along with optimal batch sizes, the 20 ms serving rhythm, why context lengths plateau near 200K, why API prices look the way they do, and even how over-trained frontier models are.

1 The Roofline Model of Decode

1.1 Setup

We estimate the time for one decoding step (generating one token for every sequence in a batch) on a cluster, e.g. an NVIDIA Blackwell NVL72 rack (72 GPUs). The model uses only two hardware numbers — **compute throughput** (FLOPs, multiplies/s) and **memory bandwidth** (BW, bytes/s) — and two model-side quantities: the **weights** and the **KV cache**. The step time is bounded below by both the compute time and the memory-fetch time:

The roofline bound

$$T \geq \max(t_{\text{compute}}, t_{\text{mem}})$$

$$t_{\text{compute}} = \frac{B \cdot N_{\text{act}}}{\text{FLOPs}} \quad t_{\text{mem}} = \underbrace{\frac{N_{\text{tot}}}{\text{BW}}}_{\text{weight fetch}} + \underbrace{\frac{B \cdot L_{\text{ctx}} \cdot \beta_{\text{tok}}}{\text{BW}}}_{\text{KV-cache fetch}}$$

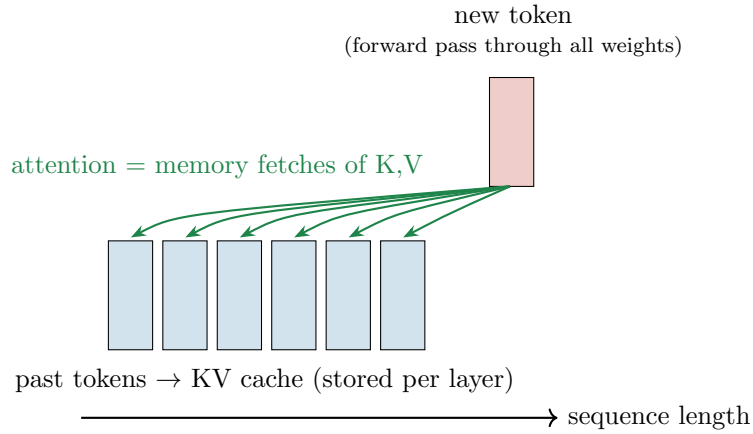
where B = batch size (number of sequences), N_{act} = active parameters, N_{tot} = total parameters, L_{ctx} = context length, β_{tok} = KV-cache bytes per token.

Notes on the terms:

- **Compute:** every batch element multiplies by every *active* parameter once. For DeepSeek-V3: $N_{\text{act}} \approx 37\text{B}$ active out of $N_{\text{tot}} \approx 700\text{B}$ total (MoE). Attention FLOPs are comparatively small and ignored.
- **Weight fetch:** *all* parameters must be read from HBM once per step — even inactive experts may be needed by some token in the batch. Crucially, this cost is **independent of B** : it is *amortized* across the batch.
- **KV fetch:** each sequence attends over its own private history, so this term is **linear in B** — it cannot be amortized.

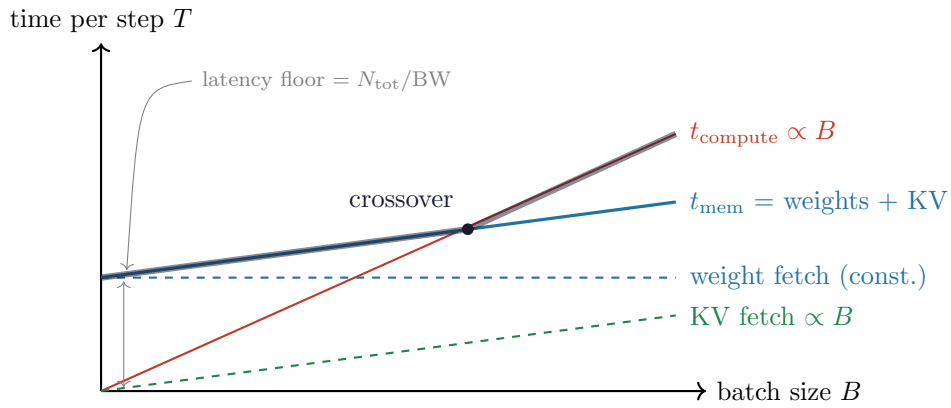
1.2 What is the KV cache?

During autoregressive decode, producing one new token requires (i) a full forward pass of that single token through all weight matrices, and (ii) attention of that token over the model’s stored internal representation of every past token — the **KV cache**. Attention during decode is dominated by *memory reads*, not matrix multiplies.



2 Latency vs. Batch Size

Plot each term of the bound against B (fixed context length):



Latency floor

For a given hardware configuration there is a hard lower bound on per-step latency:

$$T_{\min} = \frac{N_{\text{tot}}}{\text{BW}}$$

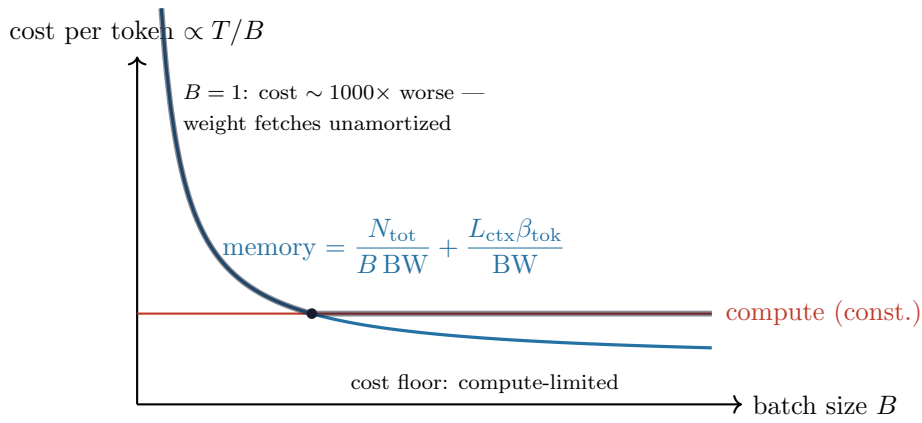
You must stream all weights from HBM through the chips once per token, no matter how small the batch. Small batches barely increase latency above this floor — which is exactly the headroom “Fast Mode” exploits.

At the crossover the system transitions from **memory-bound** (left) to **compute-bound** (right). The crossover location is sensitive to context length: longer contexts steepen the KV line and push the system toward memory-bound. When the slopes of t_{mem} and t_{compute} match, the system is simultaneously compute- and bandwidth-saturated — the ideal operating point. (Caveat: the linear-in- L_{ctx} KV fetch assumes *dense* attention; published sparse-attention schemes, e.g. DeepSeek’s, replace L_{ctx} by roughly $\sqrt{L_{\text{ctx}}}$.)

3 Cost vs. Batch Size: Why Batching Is Everything

GPU rental costs \$/s, and one step serves B tokens, so **cost per token** $\propto T/B$. Divide every curve by B :

$$\frac{t_{\text{compute}}}{B} = \frac{N_{\text{act}}}{\text{FLOPs}} \text{ (constant)}, \quad \frac{\text{KV}}{B} = \frac{L_{\text{ctx}} \beta_{\text{tok}}}{\text{BW}} \text{ (constant)}, \quad \frac{\text{weights}}{B} = \frac{N_{\text{tot}}}{B \cdot \text{BW}} \text{ (hyperbola, } \propto 1/B)$$



Intuition

At $B = 1$ you pay the entire weight-streaming bill for a single token — up to $\sim 1000\times$ the large-batch cost. As B grows the weight fetch amortizes away and cost asymptotes to a floor set by compute (plus the per-token KV fetch, which never amortizes). This answers the opening question: a “Slow Mode” beyond the crossover batch size buys almost nothing — you are already on the cost floor. Fast Mode works in reverse: run a small batch, sacrifice amortization (hence $6\times$ price), gain latency ($2.5\times$ speed).

4 The Optimal Batch Size $\approx 300\times$ Sparsity

Equate weight-fetch time with compute time (dropping the KV term for a clean answer):

$$\frac{N_{\text{tot}}}{BW} = \frac{B N_{\text{act}}}{\text{FLOPs}} \implies \underbrace{\frac{\text{FLOPs}}{BW}}_{\text{hardware}} = B \cdot \underbrace{\frac{N_{\text{act}}}{N_{\text{tot}}}}_{1/\text{sparsity}}$$

The hardware ratio is made dimensionless by counting FP4 multiplies ($\frac{1}{2}$ byte each); across A100 $\rightarrow$ H100 $\rightarrow$ B100 it has stayed remarkably stable at ≈ 300 .

Critical batch size

$$B^* \approx 300 \times \text{sparsity} = 300 \times \frac{N_{\text{tot}}}{N_{\text{act}}}$$

DeepSeek-style sparsity (32 of 256 experts active $\Rightarrow$ sparsity = 8): $B^* \approx 2,400$ sequences decoding concurrently. In practice run $2\text{--}3\times$ larger, since real efficiency is below roofline. Including the KV term pushes B^* *higher* still (KV consumes bandwidth that weight loading needs). Note B^* depends only on sparsity — not on model scale.

4.1 The 20-millisecond train schedule

A second constraint sets the step time itself: you want to *use* all the HBM capacity you bought, and reading it exactly once per forward pass takes

$$t_{\text{drain}} = \frac{\text{HBM capacity}}{\text{HBM bandwidth}} \approx 20 \text{ ms} \quad (\text{e.g. Rubin: } 288 \text{ GB} / 20 \text{ TB/s} \approx 15 \text{ ms})$$

This ratio has hovered near 15–20 ms across HBM generations. Taking longer means reading HBM twice per step — pointless, since weights and KV are read-only and needed once. So serving runs like a train timetable: **a batch departs every ~ 20 ms**, full or not. Worst-case queueing latency = miss one train + ride the next = ~ 40 ms.

4.2 System throughput

$$\text{tokens/s} = \frac{B^*}{t_{\text{drain}}} \approx 2,000 \times \frac{1}{15 \text{ ms}} \approx 2,000 \times 64 \approx 128\text{K tok/s per rack-scale system.}$$

For scale: Gemini-class global traffic was announced in the *hundreds of millions* of tokens/s — so one saturated system is $\sim 1/1000$ of a top provider. Batching economics are a real but modest centralizing force: you need roughly a thousandth of Gemini’s traffic to serve at the cost floor.

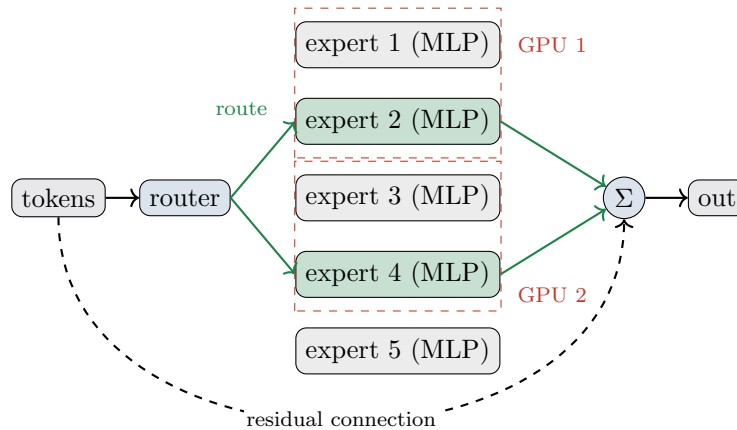
5 How Sparse Should Models Be?

Increasing sparsity (more total params, fewer active) cuts compute cost per token, but empirically the quality return is sublinear: in “Unified Scaling Laws for Routed Language Models,” a 64-expert model with 370M active parameters merely matches a 1.3B dense model — $\sim 64\times$ the parameters for $\sim 4\times$ effective capacity (older MoE recipe; DeepSeek-style fine-grained experts do better). From the systems side, doubling sparsity doubles N_{tot} (weight fetch + memory capacity) but that cost amortizes by running a bigger batch — a pure win *if* you have the users and the memory capacity. The binding constraints on sparsity are therefore:

1. **Available traffic:** $B^* \approx 300\times$ sparsity concurrent sequences must exist.
2. **Memory capacity** of the interconnect domain (next section).
3. **Model quality** degradation at extreme sparsity (empirical).

6 MoE on a Rack: Expert Parallelism and Interconnects

6.1 Anatomy of an MoE layer

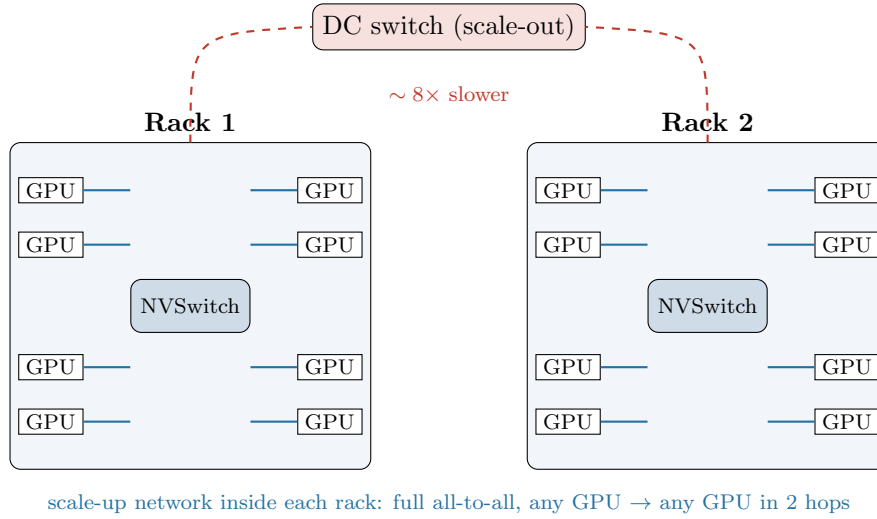


Each expert is an ordinary MLP: $W_{\text{up}} \rightarrow \sigma \rightarrow W_{\text{down}}$. Green = activated for this token (e.g. 32 of 256 in DeepSeek).

Expert parallelism (the standard, best layout): different experts live on different GPUs. E.g. DeepSeek’s 256 experts on a 72-GPU rack $\rightarrow$ use 64 GPUs, 4 experts each. The router is replicated on every GPU. Routing decisions send any token to any GPU: an **all-to-all** traffic pattern — a perfect match for a rack’s fully-switched internal network.

6.2 Scale-up vs. scale-out

A *rack* ($\sim 2\text{m}$ tall, ~ 64 GPUs; limited by power delivery, weight, cooling) contains the **scale-up network** (NVLink + NVSwitches): every GPU reaches every other in two hops at full bandwidth. Leaving the rack uses the **scale-out network** (NICs $\rightarrow$ datacenter switches), typically $\sim 8\times$ slower.



If one MoE layer spans two racks, half of all routed tokens must cross the slow link — it becomes the bottleneck. Hence: **one scale-up domain bounds the width of an expert layer**, which is what has driven NVLink domains from 8 (Hopper trays) → 72 (Blackwell rack: a form-factor/product change) → ~576 (Rubin: genuinely harder rack engineering — cable density, connector density, bend radius, weight, power, cooling). One big flat switch is precluded mostly by physical cabling congestion.

7 Pipeline Parallelism: Cutting Along Layers

Other parallelisms map racks to model dimensions: **tensor parallelism** (cut inside an expert — now mostly irrelevant, experts are small), **data parallelism**, and **pipeline parallelism** (consecutive layer blocks on different racks). The cut directions that win are the ones that mirror the architecture's own large dimensions: experts → GPUs within a rack, layers → across racks.

7.1 When is crossing racks affordable?

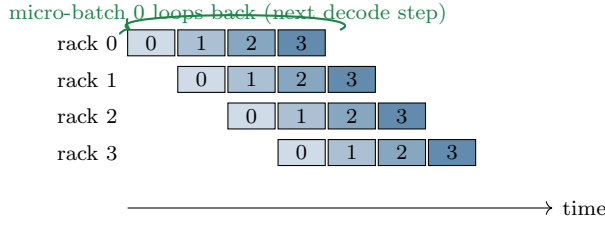
Compare time on scale-up vs. scale-out links. Per token, the all-to-all inside the rack sends to n_{exp} activated experts, twice (scatter + gather), for every one of $n_{\text{layers/stage}}$ layers; the pipeline hop sends the token *once*:

$$\frac{t_{\text{scale-up}}}{t_{\text{scale-out}}} = \frac{1}{8} \times 2 \times n_{\text{exp}} \times n_{\text{layers/stage}} \geq 1$$

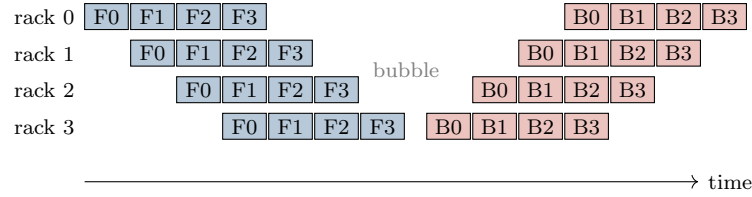
With $n_{\text{exp}} = 8\text{--}32$ this is trivially satisfied even at one layer per stage: **pipelining across racks is communication-cheap**. Caveat: each rack-to-rack hop adds latency (order ms, via NIC → top-of-rack/DC switch → NIC), and decode hops are sequential — 4 stages can add ~10 ms/token, significant against a 20 ms step.

7.2 Pipeline schedules and the bubble

Inference (forward only): no real bubble



Training: hard stop at backward pass $\Rightarrow$ bubble



In **inference**, decode loops: micro-batch 0 finishes stage 3 and immediately re-enters stage 0 for its next token. Boxes tile time perfectly — pipelining is *latency- and throughput-neutral*; “micro-batch vs. batch” is just labeling. In **training**, the backward pass must wait for the full global batch (there is an optimal batch size: smaller is better for convergence, worse for systems), creating **pipeline bubbles**; zero-bubble / 1F1B schedules interleave forward and backward to fill them.

8 Memory Capacity: Why Pipelining Doesn’t Save the KV Cache

System-wide memory demand and the per-GPU share under expert parallelism of extent E (GPUs/rack) and pipelining of extent P (racks):

$$C_{\text{mem}} = N_{\text{tot}} + B L_{\text{ctx}} \beta_{\text{tok}} \quad \Rightarrow \quad c_{\text{mem}} = \frac{N_{\text{tot}} + B L_{\text{ctx}} \beta_{\text{tok}}}{E P}$$

But keeping P stages busy in decode requires P micro-batches in flight: the global batch is $B = P \cdot b$ where $b \approx 300 \times \text{sparse}$ is the per-train micro-batch. Substituting:

The KV cache does not shard across pipeline stages

$$c_{\text{mem}} = \frac{N_{\text{tot}}}{E P} + \frac{P b L_{\text{ctx}} \beta_{\text{tok}}}{E P} = \underbrace{\frac{N_{\text{tot}}}{E P}}_{\downarrow \text{ with } P} + \underbrace{\frac{b L_{\text{ctx}} \beta_{\text{tok}}}{E}}_{\text{constant in } P}$$

Pipelining shrinks per-GPU *weight* storage but leaves per-GPU *KV* storage untouched: storing one layer’s KV instead of P layers’ is exactly cancelled by needing $P \times$ more sequences in flight. KV cannot be amortized across the batch *and* cannot be sharded across pipeline stages.

Practice (e.g. DeepSeek’s reported serving setup): **maximize expert parallelism up to the scale-up domain**, pipeline little or not at all ($P = 1$ or 2 — a trillion-param model is ~ 1 TB, and a Blackwell rack holds tens of TB, so weights already fit).

8.1 Why bigger scale-up domains mattered for model scale

Pipelining already solves weight *capacity*. What big scale-up domains really buy is **bandwidth** — the latency floor uses every GPU in the domain in parallel:

$$t_{\text{mem}} = \frac{N_{\text{tot}}}{S \times \text{BW}_{\text{GPU}}} \quad (S = \text{scale-up size})$$

Per-GPU bandwidth improves $\sim 1.5\text{--}2\times$ per generation, but S jumped $8\times$ (Hopper $\rightarrow$ Blackwell) and again with Rubin. That is what makes big- N_{tot} , long-context, low-latency serving viable — and part of why frontier total parameter counts stagnated near GPT-4 scale ($\sim 1\text{T}$, 2023) until rack-scale systems landed: $8\times\text{H100} \approx 640\text{GB}$ couldn't hold a 5T model + KV for a real batch; a Blackwell rack (10–20 TB) can. (Google's TPU pods had large scale-up domains much earlier — plausibly one ingredient in Gemini's pre-training lead, alongside MoE recipe and data.)

9 Balancing Pre-training, RL, and Inference Compute

Heuristic: a sum of opposing costs is minimized near the point where the terms are *equal* (true for $x + 1/x$, $e^x + e^{-x}$, and generally power-law-ish curves). Apply to total compute with the $6ND$ rule (forward+backward = 6 FLOPs/param/token; forward alone = 2):

$$\underbrace{6 N_{\text{act}} D_{\text{pre}}}_{\text{pre-training}} = \underbrace{(2\text{--}6) \frac{N_{\text{act}} D_{\text{RL}}}{\eta}}_{\text{RL (decode-heavy, } \eta \approx 0.3 \text{ MFU penalty)}} = \underbrace{2 N_{\text{act}} D_{\text{inf}}}_{\text{inference}}$$

N_{act} cancels; with the inefficiency factors folded in, the three *token counts* come out within small factors of each other:

$$D_{\text{pre}} \sim D_{\text{RL}} \sim D_{\text{inf}}$$

Intuition

A well-balanced frontier model should *generate* roughly as many tokens in deployment as it read in pre-training — it streams out “the sum of human knowledge” once. RL token counts sit somewhat below pre-training tokens because each RL token costs more wall-clock (decode MFU $\ll$ training MFU).

9.1 Back-solving the over-training factor

Estimate inference traffic per flagship model: $\sim 50\text{M tok/s} \times 2 \text{ months} \approx 2 \times 10^{14}$ (200T) tokens $\Rightarrow D_{\text{pre}}$ of the same order (rumored frontier pre-training: $\sim 150\text{T}$ tokens — consistent!). Chinchilla-optimal data for $N_{\text{act}} \approx 100\text{B}$ is $D_{\text{Chin}} = 20 N_{\text{act}} \approx 2\text{T}$ tokens. Hence:

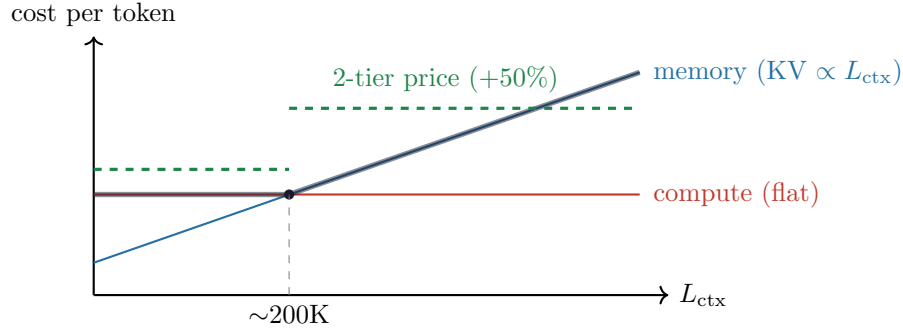
$$\text{over-training factor} = \frac{D_{\text{pre}}}{D_{\text{Chin}}} \approx \frac{200\text{T}}{2\text{T}} \approx 100\times$$

Frontier models are roughly **100 $\times$ over-trained** relative to Chinchilla — because inference (and RL) demand favors small- N_{act} , data-saturated models. (Big error bars; derate inference if the model might miss frontier and be discarded.)

10 Reading the Tea Leaves in API Pricing

10.1 Long-context surcharge (>200K tokens, +50%)

Cost vs. context length at fixed large batch: compute is flat in L_{ctx} (the quadratic attention-FLOPs term has negligible slope until millions of tokens); the KV memory fetch grows linearly. The pricing breakpoint plausibly marks the crossover:



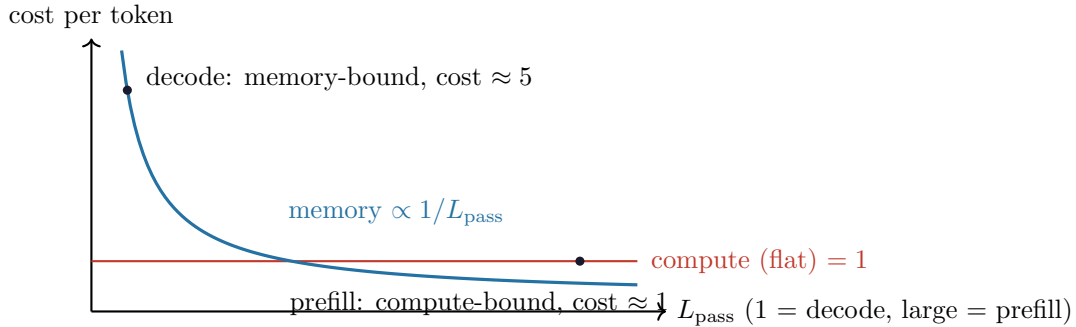
Assuming the crossover sits at $L_{\text{ctx}} = 200\text{K}$ and the weight-fetch is amortized away, equate KV fetch with compute and solve for KV bytes/token:

$$\frac{L_{\text{ctx}} \beta_{\text{tok}}}{\text{BW}} = \frac{N_{\text{act}}}{\text{FLOPs}} \Rightarrow \beta_{\text{tok}} = \frac{N_{\text{act}}}{(\text{FLOPs}/\text{BW}) \cdot L_{\text{ctx}}} \approx \frac{10^{11}}{300 \times 2 \times 10^5} \approx 1.7 \text{ kB/token}$$

Sanity check via architecture: $\beta_{\text{tok}} = n_{\text{layers,unique}} \times 2 \times d_{\text{head}} \times n_{\text{KV-heads}} \times (\text{bytes/elt})$. With a Character-AI/Gemma-style shared global context, $d_{\text{head}} = 128$, 8 KV heads: $\approx 2 \text{ kB}$. Plausible — the pricing structure leaks architecture.

10.2 Input 3–5× cheaper than output: prefill vs. decode

Treat both as a forward pass over L_{pass} tokens ($L_{\text{pass}} = 1$ is decode; large is prefill) and plot *per-token* cost T/L_{pass} : compute per token is flat in L_{pass} , while the memory term (read whole KV + weights once per pass, amortized over L_{pass} tokens) falls as $1/L_{\text{pass}}$. With flash attention, prefill's attention intermediates stay on-chip, so no extra HBM traffic.



A 5× output/input price ratio therefore directly measures how memory-bandwidth-bound decode is: **prefill is compute-limited, decode is bandwidth-limited**, with a $\sim 5\times$ gap between decode's memory time and its compute time.

10.3 Prompt caching and the memory hierarchy

A KV cache for a prefix can be (a) **rematerialized** (recompute from token IDs: cost = one forward pass of compute) or (b) **stored** in some memory tier. Each tier has a *retrieve* cost (copy into HBM) and a *hold* cost (rent on capacity $\times$ duration):

Option	Cost to retrieve	Cost to hold	Drain time $\frac{\text{capacity}}{\text{bandwidth}}$
Rematerialize	$\frac{N_{\text{act}}}{\text{FLOPs}} \times \$_{\text{GPU}}/\text{s}$	0	—
HBM	≈ 0 (already resident)	$\frac{\beta_{\text{tok}}}{C_{\text{HBM}}} \times \$_{\text{GPU}}/\text{s} \times t_{\text{hold}}$	$\sim 20 \text{ ms}$
DDR (host)	$\beta_{\text{tok}}/\text{BW}_{\text{DDR}}$	lower	$\sim 1\text{--}10 \text{ s}$
Flash	higher	lower still	$\sim 1 \text{ min}$
Spinning disk	highest	lowest	$\sim 1 \text{ hr}$

Balance rule: store in a tier whose **drain time matches your hold duration** (equalize the device-fraction used for holding vs. retrieving). API cache options of “5 minutes” vs. “1 hour” therefore likely map to **flash** and **spinning disk** respectively — yes, frontier inference plausibly touches spinning rust. Cache-hit tokens priced $\sim 10\times$ below remat (\$5/M base vs. \$6.25/M to write 5-min cache) reflect skipping the forward-pass compute.

11 Why Context Lengths Plateau Near 100–200K

Two costs grow with context: attention *compute* (mild slope; matters only at millions of tokens) and *memory bandwidth + capacity* for the KV cache (the binding one). Context lengths jumped 8K $\rightarrow$ 100–200K (GPT-3 era $\rightarrow$ GPT-4 era) then stalled for years: evidence that 200K sits at the balanced-cost point and HBM bandwidth is not improving fast enough to move it. Sparse attention ($L_{\text{ctx}} \rightarrow \sqrt{L_{\text{ctx}}}$ -ish) is the main escape hatch but trades quality if pushed too far — and may already be priced in. Implication: if “in-context learning is enough for AGI” (the 100M-token-context employee), some piece of ML infra — not model architecture alone — has to break the KV memory wall first.

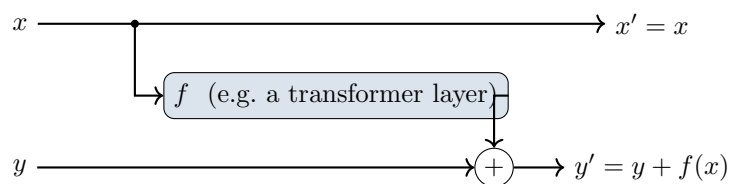
12 Bonus: Ciphers, Feistel Networks, and Reversible Transformers

Cryptographic ciphers and neural nets share a mixing/scrambling skeleton but with opposite goals: ciphers make structure look random and *want* the avalanche property (tiny input change $\rightarrow$ huge output change); neural nets extract structure from apparent randomness and engineer *tame* derivatives (residual connections, LayerNorm) so gradient descent works. Differential cryptanalysis is literally differentiating a cipher over $\mathbb{F}_2$; adversarial examples are the avalanche property showing up where it’s unwanted.

12.1 One real technology transfer: the Feistel construction

Build an *invertible* map from a non-invertible f :

$$(x, y) \mapsto (x, z = y + f(x)) \quad \text{inverse: } x = x, \quad y = z - f(x)$$



RevNets (2017) apply this to deep nets: with the swap $(x, y) \rightarrow (y + f(x), x)$ each block is a residual connection reaching *two layers back*, and the whole network is invertible. Payoff in training: instead of storing every layer’s activations for the backward pass (HBM footprint linear in depth), **rematerialize activations by running the network backwards in lockstep** — spend compute to save memory. Note the symmetry: RevNets trade compute for memory; the KV cache trades memory for compute. Given today’s hardware ratios, spending memory to save compute is usually the profitable direction — which is, full circle, why the memory wall is the story of this whole lecture.

One-page summary

- Decode time $\geq \max(BN_{\text{act}}/\text{FLOPs}, (N_{\text{tot}} + BL_{\text{ctx}}\beta_{\text{tok}})/\text{BW})$; weights amortize over batch, KV never does.
- Latency floor = N_{tot}/BW ; cost floor = $N_{\text{act}}/\text{FLOPs}$ per token. Fast Mode = small batch (pay for lost amortization); Slow Mode beyond B^* buys nothing.
- $B^* \approx 300 \times \text{sparsity}$ ($\approx 2\text{--}3\text{K}$ sequences); batches depart every $\sim 20\text{ ms}$ (HBM drain time); $\sim 128\text{K}$ tok/s per system.
- MoE $\Rightarrow$ all-to-all $\Rightarrow$ expert parallelism within one scale-up domain; pipeline across racks is cheap on bandwidth but adds decode latency, shards weights, and *cannot* shard KV.
- Big scale-up domains matter for *bandwidth* (latency, long context), not capacity — this gated model-size scaling until Blackwell-class racks.
- Balanced frontier compute $\Rightarrow D_{\text{pre}} \sim D_{\text{RL}} \sim D_{\text{inf}} \Rightarrow \sim 100\times$ over-trained vs. Chinchilla.
- API prices leak physics: 200K context tier $\rightarrow \beta_{\text{tok}} \approx 2\text{ kB}$; output/input = $5\times \rightarrow$ decode is $5\times$ memory-bound; cache durations $\rightarrow$ flash and spinning disk tiers.